Module 8: Portfolio Milestone Part 2

Arianna Fakhri

CSU Global

CSC450-1

Professor Reginald Halestine

Pseudocode:

```
BEGIN PROGRAM
DEFINE countUp FUNCTION
        FOR number = 0 to 20
                DISPLAY “Thread 1 counting up: “, number
        END FOR
END FUNCTION

DEFINE countDown FUNCTION
        FOR number = 20 DOWN TO 0
                DISPLAY “Thread 2 counting down: “, number
        END FOR
END FUNCTION

BEGIN MAIN
        CREATE firstThread using countUp
        CREATE secondThread using countDown

        TRY
                START firstThread
                WAIT for firstThread to finish

                START secondThread
                WAIT for secondThread to finish

                DISPLAY “Both threads have finished.”
        CATCH thread interruption
                RESTORE interruption status
                DISPLAY error message
        END TRY
END MAIN

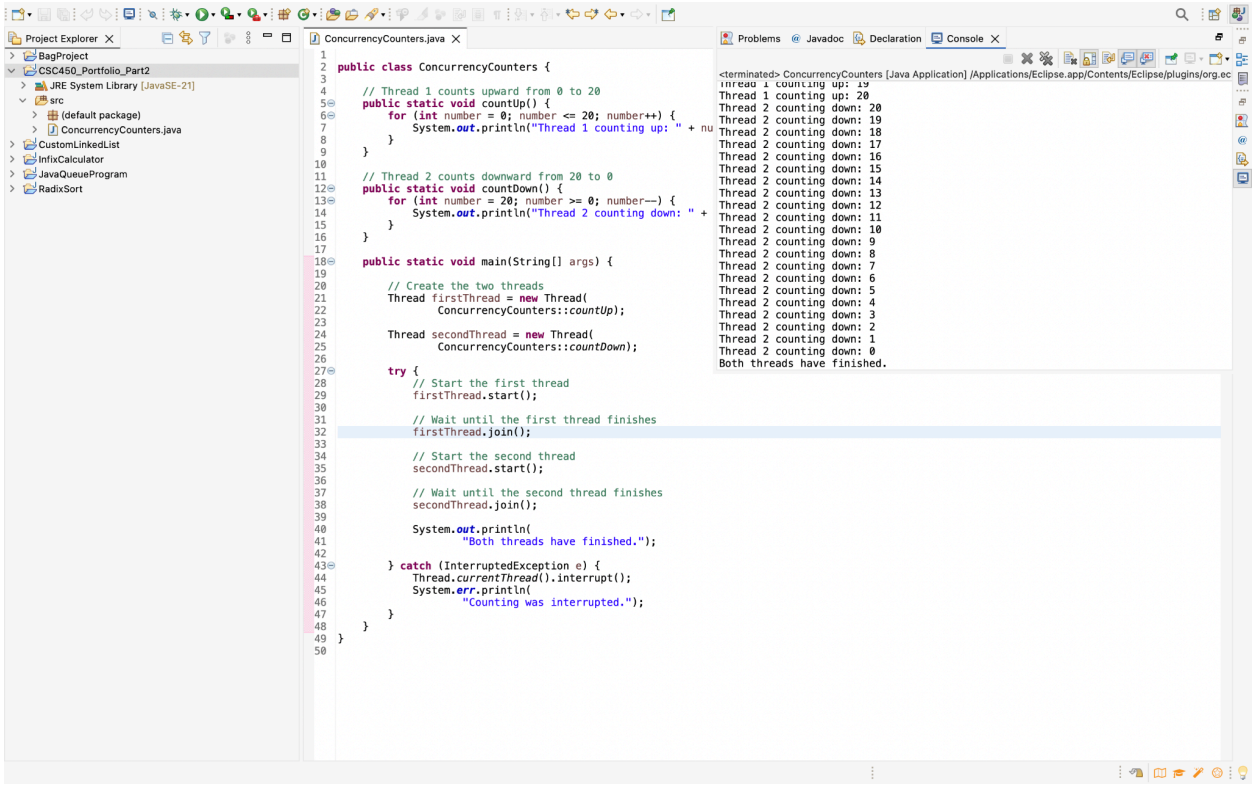
END PROGRAM
```

Source code:

```
public class ConcurrencyCounters {
  // Thread 1 counts upward from 0 to 20
  public static void countUp() {
     for (int number = 0; number <= 20; number++) {
        System.out.println("Thread 1 counting up: " + number);
     }
```

```
    }
    // Thread 2 counts downward from 20 to 0
    public static void countDown() {
        for (int number = 20; number >= 0; number--) {
            System.out.println("Thread 2 counting down: " + number);
        }
    }
    public static void main(String[] args) {
        // Create the two threads
        Thread firstThread = new Thread(
                ConcurrencyCounters::countUp);
        Thread secondThread = new Thread(
                ConcurrencyCounters::countDown);
        try {
            // Start the first thread
            firstThread.start();
            // Wait until the first thread finishes
            firstThread.join();
            // Start the second thread
            secondThread.start();
            // Wait until the second thread finishes
            secondThread.join();
            System.out.println(
                    "Both threads have finished.");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.err.println(
                    "Counting was interrupted.");
        }
    }
}
```

Screenshots:



Performance Implementation Comparison:

For my Portfolio Project, I created two applications that demonstrate concurrency concepts, one using Java and the other using C++. Both programs use two separate threads to perform counting operations. The first thread counts upward from 0 to 20, and once it finishes, the second thread counts backward from 20 to 0. Although both applications perform the same task, Java and C++ have different approaches to thread management, performance, memory usage, and security. Comparing the two programs helped me better understand how these differences can affect the reliability and efficiency of an application.

Concurrency allows programs to manage multiple tasks using threads, which can improve the performance depending on the application. However, concurrency can also introduce performance issues, such as additional memory usage, thread creation overhead, and synchronization delays. These issues become especially important when applications use multiple threads that need to access shared resources or execute in a specific order. In my C++ program, I used std::thread to create two counting threads. In Java, I used the Thread class and the start() method to begin execution. Both programs use the join() method to make sure the first thread finishes before the second begins. According to cppreference.com, the join() method blocks the calling thread until the selected thread completes. This allows my applications to

control the order of execution and produce consistent results. While using join() helps prevent the two counters from running out of order, it also means the threads cannot execute simultaneously. Because of this, neither program benefits from parallel execution. For a simple counting application like mine, regular loops would likely be more efficient since creating threads requires additional system resources. However, implementing the threads helped demonstrate how synchronization works and why it is important when tasks depend on one another.

Another difference between the two languages is how they execute programs. C++ is typically compiled directly into native machine code, while Java runs through the Java Virtual Machine. C++ can have lower runtime overhead, but Java uses optimizations such as just in time compilation to improve performance. Because both of my applications only count between 0 and 20, the difference in actual counting performance would likely be very small. Thread creating, console output, and runtime startup may have a greater impact on execution time. Without conducting controlled performance tests, it's hard to tell which version is faster. However, I would expect both programs to complete the required operations quickly. For larger applications with more complex calculations, the performance differences between Java and C++ could become more noticeable.

String handling is another important consideration when developing secure applications. Improperly handling strings can introduce vulnerabilities such as buffer overflows, memory corruption, and problems caused by unvalidated user input. In C++, developers can work with fixed-size character arrays and manually managed memory. If a program attempts to store more characters than an array can hold, it may cause a buffer overflow. This can lead to unexpected behavior, crashes, or security vulnerabilities. Using safer alternatives, such as std::string, and validating input can help reduce these risks. Java provides additional protection through its automatic memory management and built in String class. Java strings are immutable, meaning their contents cannot be changed after they are created. Java also performs runtime checks that help prevent certain memory related vulnerabilities. According to Oracle (2025), Java includes features such as automatic memory management, type safety, and array bounds checking to reduce common programming errors. In both of my applications, I used predefined strings to display messages such as "Thread 1 counting up" and "Thread 2 counting down". Since neither program accepts user input or modifies the displayed strings, the risk of string related vulnerabilities is limited. If I expanded either application to accept input from users, I would need to validate the input before processing it. This could include checking string lengths, rejecting unexpected characters when appropriate, and using safe string handling methods. Although Java provides stronger built in memory protections, both languages still require secure programming practices to prevent vulnerabilities.

Choosing appropriate data types is important because incorrect data handling can lead to problems such as integer overflow, memory errors, and unexpected results. Data types determine what kind of information a variable can store and how much memory is available for that information. For both my Java and C++ applications, I used the int data type to store the counter values. Since the programs only count between 0 and 20, these values are well within the supported range of an integer. This makes int an appropriate choice for both applications. One difference between the languages is how integer values are represented. Java's int is always a signed 32-bit integer, while the size of an int in C++ can depend on the compiler and the system. Another difference is how integer overflow is handled. Java uses defined

wraparound behavior for normal integer arithmetic, while signed integer overflow in C++ results in undefined behavior. Although overflow is not a concern in my current applications. It would be important to consider if the programs were expanded to process larger values. Another security concern with concurrency is race conditions. A race condition can occur when multiple threads access shared data at the same time and at least one thread modifies that data without proper synchronization. This can produce incorrect or unpredictable results. Both of my applications avoid this problem because each thread uses its own local counter variable. The program also uses join() to ensure the threads execute in the required order. If I expanded either application to allow multiple threads to modify shared variables, I would need to implement additional synchronization methods. In Java, I could use synchronized blocks or atomic variables to protect shared data. In C++ I could use mutexes or atomic variables for the same purpose. These features help maintain data consistency. But they must be implemented correctly to avoid additional performance problems or deadlocks.

When comparing the overall security of Java and C++, I would consider Java the less vulnerable option by default, mainly because of its automatic memory management and built in runtime protections. Java manages most memory allocation and cleanup through the JVM, reducing the risk of common memory management mistakes. It also includes array bounds checking the type safety, which can help prevent certain vulnerabilities. Oracle (2025) explains that these features provide protection against several memory related security problems that can occur in languages such as C and C++. C++ gives programmers more direct control over memory and system resources. This can be beneficial when developing applications that require low level operations or specific performance optimizations. However, it also places more responsibility on developers to avoid issues such as invalid memory access, buffer overflows, and improper resource management. This does not mean Java applications are completely secure or that C++ applications are unsafe. Java programs can still experience vulnerabilities related to improper input validation, race conditions, and incorrect application logic. Similarly, C++ developers can reduce security risks by using modern language features, safe libraries, and proper memory management practices. For my particular program, both implementations have limited security risks because they only perform simple counting operations. Neither application accepts external input, accesses sensitive information, or allows the counting threads to modify shared variables simultaneously. However, if these applications were expanded into more complex systems, I believe Java would provide an advantage when it comes to reducing common memory-related vulnerabilities. C++ may provide advantages in situations where direct control over system resources is necessary, but it generally requires more attention to memory safety.

Overall, developing my Java and C++ concurrency applications helped me understand how both languages manage threads and how their differences can affect performance and security. Both programs successfully perform the same counting operations and use join() to ensure the threads execute in the correct order. Although C++ can provide performance advantages in certain applications, Java offers additional memory safety protections that can reduce common security risks. For my specific programs, neither language has a demonstrated performance advantage because the operations are simple and the threads execute sequentially. If I were developing a larger application, I would prefer to use Java. However C++ would still be a useful option for applications that require more direct control over the system.

This project showed me that choosing the right programming language depends on the applications requirements and how carefully performance and security are managed.

References:

STD::Thread::Join - cppreference.com. cppreference.com. (n.d.). https://en.cppreference.com/cpp/thread/thread/join

Oracle. (2019, March 4). *Secure Coding Guidelines for Java SE*. Secure coding guidelines for java SE. https://www.oracle.com/java/technologies/javase/seccodeguide.html